

Lab 12: SQL lab12.zip (lab12.zip)

Due by 11:59pm on Wednesday, December 4.

Starter Files

Download [lab12.zip](#) (lab12.zip).

Required Questions

SQL

A `SELECT` statement describes an output table based on input rows. To write one:

1. Describe the **input rows** using `FROM` and `WHERE` clauses.
2. **Group** those rows and determine which groups should appear as output rows using `GROUP BY` and `HAVING` clauses.
3. Format and order the **output rows** and columns using `SELECT` and `ORDER BY` clauses.

`SELECT (Step 3) FROM (Step 1) WHERE (Step 1) GROUP BY (Step 2) HAVING (Step 2) ORDER BY (Step 3);`

Step 1 may involve joining tables (using commas) to form input rows that consist of two or more rows from existing tables.

The `WHERE`, `GROUP BY`, `HAVING`, and `ORDER BY` clauses are optional.

Consult the drop-down for a refresher on SQL. It's okay to skip directly to the questions and refer back here should you get stuck.

SQL

SQL Basics

Creating Tables

You can create SQL tables either from scratch or from existing tables.

The following statement creates a table by specifying column names and values without referencing another table. Each `SELECT` clause specifies the values for one row, and `UNION` is used to join rows together. The `AS` clauses give a name to each column; it need not be repeated in subsequent rows after the first.

```
CREATE TABLE [table_name] AS
  SELECT [val1] AS [column1], [val2] AS [column2], ... UNION
  SELECT [val3]           , [val4]           , ... UNION
  SELECT [val5]           , [val6]           , ...;
```

Let's say we want to make the following table called `big_game` which records the scores for the Big Game each year. This table has three columns: `berkeley`, `stanford`, and `year`.

berkeley	stanford	year
30	7	2002
28	16	2003
17	38	2014

We could do so with the following `CREATE TABLE` statement:

```
CREATE TABLE big_game AS
  SELECT 30 AS berkeley, 7 AS stanford, 2002 AS year UNION
  SELECT 28,           16,           2003      UNION
  SELECT 17,           38,           2014;
```

Selecting From Tables

More commonly, we will create new tables by selecting specific columns that we want from existing tables by using a `SELECT` statement as follows:

```
SELECT [columns] FROM [tables] WHERE [condition] ORDER BY [columns] LIMIT [limit];
```

Let's break down this statement:

- `SELECT [columns]` tells SQL that we want to include the given columns in our output table; `[columns]` is a comma-separated list of column names, and `*` can be used to select all columns
- `FROM [table]` tells SQL that the columns we want to select are from the given table
- `WHERE [condition]` filters the output table by only including rows whose values satisfy the given `[condition]`, a boolean expression

- `ORDER BY [columns]` orders the rows in the output table by the given comma-separated list of columns
- `LIMIT [limit]` limits the number of rows in the output table by the integer `[limit]`

Here are some examples:

Select all of Berkeley's scores from the `big_game` table, but only include scores from years past 2002:

```
sqlite> SELECT berkeley FROM big_game WHERE year > 2002;
28
17
```

Select the scores for both schools in years that Berkeley won:

```
sqlite> SELECT berkeley, stanford FROM big_game WHERE berkeley > stanford;
30|7
28|16
```

Select the years that Stanford scored more than 15 points:

```
sqlite> SELECT year FROM big_game WHERE stanford > 15;
2003
2014
```

SQL operators

Expressions in the `SELECT`, `WHERE`, and `ORDER BY` clauses can contain one or more of the following operators:

- comparison operators: `=`, `>`, `<`, `<=`, `>=`, `<>` or `!=` ("not equal")
- boolean operators: `AND`, `OR`
- arithmetic operators: `+`, `-`, `*`, `/`
- concatenation operator: `||`

Output the ratio of Berkeley's score to Stanford's score each year:

```
sqlite> select berkeley * 1.0 / stanford from big_game;
0.447368421052632
1.75
4.28571428571429
```

Output the sum of scores in years where both teams scored over 10 points:

```
sqlite> select berkeley + stanford from big_game where berkeley > 10 and stanford > 10;
55
44
```



Output a table with a single column and single row containing the value "hello world":

```
sqlite> SELECT "hello" || " " || "world";  
hello world
```

Joins

To select data from multiple tables, we can use joins. There are many types of joins, but the only one we'll worry about is the inner join. To perform an inner join on two or more tables, simply list them all out in the FROM clause of a SELECT statement:

```
SELECT [columns] FROM [table1], [table2], ... WHERE [condition] ORDER BY [columns] LIMIT
```

We can select from multiple different tables or from the same table multiple times.

Let's say we have the following table that contains the names of head football coaches at Cal since 2002:

```
CREATE TABLE coaches AS  
  SELECT "Jeff Tedford" AS name, 2002 as start, 2012 as end UNION  
  SELECT "Sonny Dykes"      , 2013      , 2016      UNION  
  SELECT "Justin Wilcox"    , 2017      , null;
```

When we join two or more tables, the default output is a cartesian product (https://en.wikipedia.org/wiki/Cartesian_product). For example, if we joined big_game with coaches, we'd get the following:

berkeley	stanford	year	name	start	end
30	7	2002	Jeff Tedford	2002	2012
28	16	2003	Sonny Dykes	2013	2016
17	38	2014	Justin Wilcox	2017	null

berkeley	stanford	year	name	start	end
30	7	2002	Jeff Tedford	2002	2012
30	7	2002	Sonny Dykes	2013	2016
30	7	2002	Justin Wilcox	2017	null
28	16	2003	Jeff Tedford	2002	2012
28	16	2003	Sonny Dykes	2013	2016
28	16	2003	Justin Wilcox	2017	null
17	38	2014	Jeff Tedford	2002	2012
17	38	2014	Sonny Dykes	2013	2016
17	38	2014	Justin Wilcox	2017	null

If we want to match up each game with the coach that season, we'd have to compare columns from the two tables in the WHERE clause:

```
sqlite> SELECT * FROM big_game, coaches WHERE year >= start AND year <= end;
17|38|2014|Sonny Dykes|2013|2016
28|16|2003|Jeff Tedford|2002|2012
30|7|2002|Jeff Tedford|2002|2012
```

The following query outputs the coach and year for each Big Game win recorded in `big_game` :

```
sqlite> SELECT name, year FROM big_game, coaches
...>      WHERE berkeley > stanford AND year >= start AND year <= end;
Jeff Tedford|2003
Jeff Tedford|2002
```

In the queries above, none of the column names are ambiguous. For example, it is clear that the `name` column comes from the `coaches` table because there isn't a column in the `big_game` table with that name. However, if a column name exists in more than one of the tables being joined, or if we join a table with itself, we must disambiguate the column names using *aliases*.

For examples, let's find out what the score difference is for each team between a game in `big_game` and any previous games. Since each row in this table represents one game, in order to compare two games we must join `big_game` with itself:

```
sqlite> SELECT b.Berkeley - a.Berkeley, b.Stanford - a.Stanford, a.Year, b.Year
...>      FROM big_game AS a, big_game AS b WHERE a.Year < b.Year;
-11|22|2003|2014
-13|21|2002|2014
-2|9|2002|2003
```

In the query above, we give the alias `a` to the first `big_game` table and the alias `b` to the second `big_game` table. We can then reference columns from each table using dot notation with the aliases, e.g. `a.Berkeley`, `a.Stanford`, and `a.Year` to select from the first table.

SQL Aggregation

Previously, we have been dealing with queries that process one row at a time. When we join, we make pairwise combinations of all of the rows. When we use `WHERE`, we filter out certain rows based on the condition. Alternatively, applying an aggregate function (http://www.sqlite.org/lang_aggfunc.html) such as `MAX(column)` combines the values in multiple rows.

By default, we combine the values of the *entire* table. For example, if we wanted to count the number of flights from our `flights` table, we could use:

```
sqlite> SELECT COUNT(*) from FLIGHTS;
13
```

What if we wanted to group together the values in similar rows and perform the aggregation operations within those groups? We use a `GROUP BY` clause.

Here's another example. For each unique departure, collect all the rows having the same departure airport into a group. Then, select the `price` column and apply the `MIN` aggregation to recover the price of the cheapest departure from that group. The end result is a table of departure airports and the cheapest departing flight.

```
sqlite> SELECT departure, MIN(price) FROM flights GROUP BY departure;
AUH|932
LAS|50
LAX|89
SEA|32
SFO|40
SLC|42
```

Just like how we can filter out rows with `WHERE`, we can also filter out groups with `HAVING`. Typically, a `HAVING` clause should use an aggregation function. Suppose we want to see all airports with at least two departures:

```
sqlite> SELECT departure FROM flights GROUP BY departure HAVING COUNT(*) >= 2;
LAX
SFO
SLC
```

Note that the `COUNT(*)` aggregate just counts the number of rows in each group. Say we want to count the number of *distinct* airports instead. Then, we could use the following query:

```
sqlite> SELECT COUNT(DISTINCT departure) FROM flights;
6
```

This enumerates all the different departure airports available in our `flights` table (in this case: SFO, LAX, AUH, SLC, SEA, and LAS).

Usage

First, check that a file named `sqlite_shell.py` exists alongside the assignment files. If you don't see it, or if you encounter problems with it, scroll down to the Troubleshooting section to see how to download an official precompiled SQLite binary before proceeding.

You can start an interactive SQLite session in your Terminal or Git Bash with the following command:

```
python3 sqlite_shell.py
```

While the interpreter is running, you can type `.help` to see some of the commands you can run.

To exit out of the SQLite interpreter, type `.exit` or `.quit` or press `Ctrl-C`. Remember that if you see `...>` after pressing enter, you probably forgot a `;`.

You can also run all the statements in a `.sql` file by doing the following: (Here we're using the `lab13.sql` file as an example.)

1. Runs your code and then exits SQLite immediately afterwards.

```
python3 sqlite_shell.py < lab13.sql
```

2. Runs your code and then opens an interactive SQLite session, which is similar to running Python code with the interactive `-i` flag.

```
python3 sqlite_shell.py --init lab13.sql
```

Final Exam Rooms

The `finals` table has columns `hall` (strings) and `course` (strings), and has rows for the lecture halls in which a course is holding its final exam.

The `sizes` table has columns `room` (strings) and `seats` (numbers), and has one row per unique room on campus containing the number of seats in that room. All lecture halls are rooms.

```
CREATE TABLE finals AS
SELECT "RSF" AS hall, "61A" as course UNION
SELECT "Wheeler" , "61A" UNION
SELECT "Pimentel" , "61A" UNION
SELECT "Li Ka Shing", "61A" UNION
SELECT "Stanley" , "61A" UNION
SELECT "RSF" , "61B" UNION
SELECT "Wheeler" , "61B" UNION
SELECT "Morgan" , "61B" UNION
SELECT "Wheeler" , "61C" UNION
SELECT "Pimentel" , "61C" UNION
SELECT "Soda 310" , "61C" UNION
SELECT "Soda 306" , "10" UNION
SELECT "RSF" , "70";
```

```
CREATE TABLE sizes AS
SELECT "RSF" AS room, 900 as seats UNION
SELECT "Wheeler" , 700 UNION
SELECT "Pimentel" , 500 UNION
SELECT "Li Ka Shing", 300 UNION
SELECT "Stanley" , 300 UNION
SELECT "Morgan" , 100 UNION
SELECT "Soda 306" , 80 UNION
SELECT "Soda 310" , 40 UNION
SELECT "Soda 320" , 30;
```

Q1: Room Sharing

Create a sharing table with two columns:

- course (strings): The name of a course
- shared (numbers): The number of rooms the course uses that are also shared with other courses

Include a row for every course that uses at least one room also occupied by another course. Do not include it in the table if a course does not share any rooms.

Reminder: COUNT(DISTINCT x) evaluates to the number of distinct values that appear in column x for a group.

Your query should work correctly for any data that might appear in the finals and sizes table, but for the example above the result should be:


```

+-----+-----+
| course | shared |
+-----+-----+
| 61A    | 3      |
| 61B    | 2      |
| 61C    | 2      |
| 70     | 1      |
+-----+-----+

```

Hint: Template

```

SELECT _.course, COUNT(DISTINCT _____) AS shared
FROM finals AS a, finals AS b
WHERE _____ GROUP BY _____;

```

1. Use FROM and WHERE to create a row for each instance of two courses sharing a final room.
2. Use GROUP BY to create one group for each course.
3. Use SELECT to compute the total number of rooms for that course which are also used by another course

```

CREATE TABLE sharing AS
SELECT "REPLACE THIS LINE WITH YOUR SOLUTION";

```

Use Ok to test your code:

```
python3 ok -q sharing
```



Q2: Two Rooms

Create a pairs table with one column:

- rooms (strings): A string describing pairs of rooms that together have at least 1,000 seats

Each string should 1) List the two room names in alphabetical order 2) State the combined number of seats in the two rooms

Only include pairs where the total seats are at least 1,000. Rows should appear in decreasing order of the total seats.

Your query should work correctly for any data that might appear in the finals and sizes table, but for the example above the result should be:

```

+-----+
| rooms                                     |
+-----+
| RSF and Wheeler together have 1600 seats |
| Pimentel and RSF together have 1400 seats |
| Li Ka Shing and RSF together have 1200 seats |
| Pimentel and Wheeler together have 1200 seats |
| RSF and Stanley together have 1200 seats |
| Li Ka Shing and Wheeler together have 1000 seats |
| Morgan and RSF together have 1000 seats |
| Stanley and Wheeler together have 1000 seats |
+-----+

```

Hint: When adding numbers and including the result in a string, put parentheses around the arithmetic:

'one plus two is ' || (1 + 2)

Hint: Template

SELECT __ **|| 'and ' ||** ' together have ' || () **|| ' seats' AS rooms**

FROM sizes AS a, sizes AS b WHERE _____
ORDER BY _____ DESC;

1. Use FROM and WHERE to create a row for each pair of different rooms (in alphabetical order) with at least 1,000 seats total.
2. No grouping is needed
3. Use SELECT to compute the total number of rooms for that course which are also used by another course

CREATE TABLE pairs AS
SELECT "REPLACE THIS LINE WITH YOUR SOLUTION";

Use Ok to test your code:

python3 ok -q pairs



Q3: (OPTIONAL) Big Courses

Create a big table with one column:

- course (strings): The name of a course

Include only courses where the total number of seats across all their final exam rooms is at least 1,000. Each course should appear in its own row.

Your query should work correctly for any data that might appear in the `finals` and `sizes` table, but for the example above the result should be:

```
+-----+
| course |
+-----+
| 61A    |
| 61B    |
| 61C    |
+-----+
```

Hint: Template

```
SELECT _____ FROM _____ WHERE _____ GROUP BY _____ HAVING _____;
```

1. Use `FROM` and `WHERE` to combine the information in the `finals` and `sizes` tables.
2. Use `GROUP BY` and `HAVING` to create one group for each course that has at least 1,000 seats.
3. Use `SELECT` to put the name of the course in the output.

```
CREATE TABLE big AS
SELECT "REPLACE THIS LINE WITH YOUR SOLUTION";
```

Use Ok to test your code:

```
python3 ok -q big
```



Q4: (OPTIONAL) Seats Remaining

Create a `remaining` table with two columns:

- `course` (strings): The name of a course
- `remaining` (numbers): The total number of seats in all rooms used for that course excluding the room with the largest number of seats

Include one row for each course and for each course, sum the seats from all but the largest room.

Your query should work correctly for any data that might appear in the `finals` and `sizes` table, but for the example above the result should be:

```

+-----+-----+
| course | remaining |
+-----+-----+
| 10     | 0         |
| 61A    | 1800      |
| 61B    | 800       |
| 61C    | 540       |
| 70     | 0         |
+-----+-----+

```

Hint: Template

```

SELECT course, _____ AS remaining
FROM _____ WHERE _____ GROUP BY _____;

```

1. Use FROM and WHERE to combine the information in the finals and sizes tables.
2. Use GROUP BY to create one group for each course.
3. Use SELECT to compute the total number of seats in all final rooms for that course except the largest one.

```

CREATE TABLE remaining AS
SELECT "REPLACE THIS LINE WITH YOUR SOLUTION";

```

Use Ok to test your code:

```
python3 ok -q remaining
```



Check Your Score Locally

You can locally check your score on each question of this assignment by running

```
python3 ok --score
```

This does NOT submit the assignment! When you are satisfied with your score, submit the assignment to Gradescope to receive credit for it.

Submit Assignment

If you are in a regular section of CS 61A, fill out this lab attendance and feedback form (<https://forms.gle/dHxj8gttNWRy6Ptm9>). (If you are in the mega section, you don't need to fill out the form.)

Then, submit this assignment by uploading any files you've edited **to the appropriate Gradescope assignment**. Lab 00 ([../lab00/#submit-with-gradescope](https://gradescope.com/submissions/61a/lab00/#submit-with-gradescope)) has detailed instructions.